

ETUDE DES CHOIX ARCHITECTURAUX CÔTÉ SERVEUR

Chapitre 2 – Durée 270 minutes



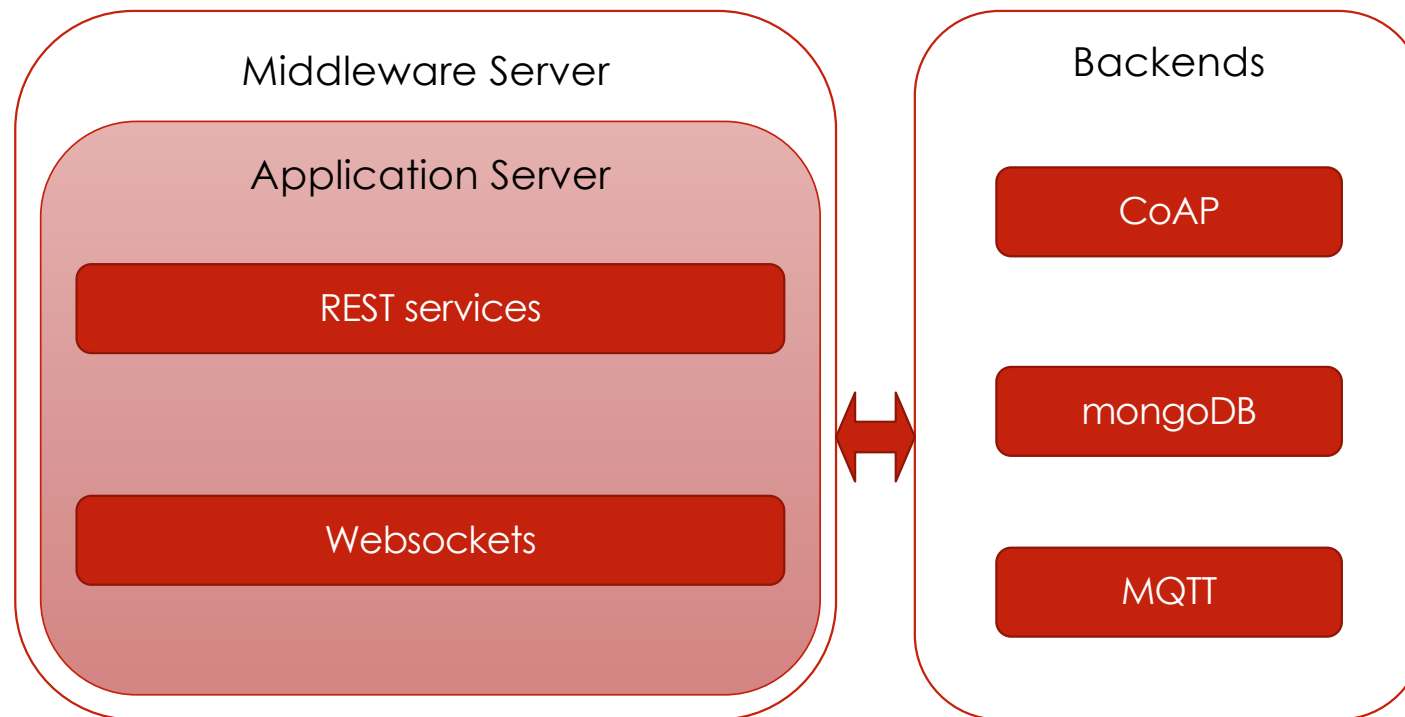
OBJECTIFS

- ✓ Etudier et comparer deux architectures pour un serveur Middleware M2M / IoT
- ✓ Installation et configuration des logiciels pour les deux architectures
- ✓ Etude de cas pour chaque architecture
- ✓ Etude comparative des deux cas

AGENDA

- ✓ Serveur Middleware pour applications mobiles M2M / IoT
- ✓ Installation et configuration de mongoDB
- ✓ Installation et configuration de Node.js
- ✓ Installation et configuration du serveur d'application Java EE Wildfly
- ✓ Etude de cas : Node.js + mongoose JS + mongoDB
- ✓ Etude de cas : Wildfly + Jakarta NoSQL/Jakarta Data + mongoDB
- ✓ Comparaison entre les deux cas serveur
- ✓ Cloud of Things

SERVEUR MIDDLEWARE POUR APPLICATIONS MOBILES M2M / IoT



INSTALLATION ET CONFIGURATION DE mongoDB

- ✓ Télécharger MongoDB Community Edition
 - ✓ <https://www.mongodb.com/download-center/community>
- ✓ Installation et configuration sous Windows
 - ✓ <http://docs.mongodb.org/manual/tutorial/install-mongodb-on-windows/>
- ✓ Installation et configuration sous OS X
 - ✓ <http://docs.mongodb.org/manual/tutorial/install-mongodb-on-os-x/>
- ✓ Installation et configuration sous Linux
 - ✓ <http://docs.mongodb.org/manual/administration/install-on-linux/>

INSTALLATION ET CONFIGURATION DE NODE.JS

✓ Télécharger & Installer Node.js (version LTS recommandée)

✓ <https://nodejs.org/en/download/>

✓ Installer Mongoose JS

✓ <https://mongoosejs.com/docs/index.html>

✓ Installer Express JS (REST services)

✓ <https://expressjs.com>

✓ Installer Socket.io ou ws (Websockets)

✓ <https://socket.io/download/>

| <https://www.npmjs.com/package/ws>

✓ <https://socket.io/get-started/chat/>

| <https://github.com/websockets/ws>

INSTALLATION ET CONFIGURATION DU SERVEUR D'APPLICATION JAVA EE

- ✓ Télécharger Wildfly 29.0.1.Final (Jakarta EE 10)
 - ✓ <https://wildfly.org/downloads/>
- ✓ Installer Wildfly 29.0.1.Final
 - ✓ <https://webdev.jhuep.com/~jcs/ejava-javaee/coursedocs/content/html/devenv-jboss-setup.html>
 - ✓ [https://docs.wildfly.org/20/Getting Started Guide.html](https://docs.wildfly.org/20/Getting%20Started%20Guide.html)
- ✓ Jakarta NoSql / Jakarta Data
 - ✓ <https://dzone.com/articles/jakarta-nosql-100-b5-how-to-make-your-life-easier>
- ✓ Installer Wildfly en tant que service:
 - ✓ <https://www.mastertheboss.com/jbossas/jboss-configuration/installing-wildfly-on-windows/>
 - ✓ <https://linux.how2shout.com/install-wildfly-application-server-on-ubuntu-20-04-22-04-lts/>

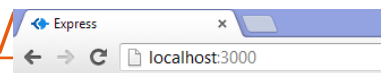
ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Dans « Node.js command prompt », taper les commandes suivantes :

```
> npm install express -g  
> npm install express-generator -g  
> express --version  
> express -e blogApp  
> cd blogApp && npm install  
> DEBUG=blogapp:* npm start    (set DEBUG=blogapp:* & npm start)@Win
```

✓ Ouvrir le navigateur et taper l'adresse : « <http://localhost:3000/> »

```
➤ wget http://localhost:3000  
➤ htmltree index.html
```



Express

Welcome to Express

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Arborescence de l'application générée :

- ✓ app.js

(Point d'entrée – lancé par ./bin/www)

- ✓ **bin**

- ✓ www

(Fichier JavaScript lançant le serveur web sur le port 3000)

- ✓ **node_modules**

- ✓ package.json

(Fichier contenant un objet JSON de configuration)
(ressources web publiques)

- ✓ **public**

- ✓ **images**

- ✓ **javascripts**

- ✓ **stylesheets**

- ✓ style.css

(Feuille de style principale)

- ✓ **routes**

- ✓ index.js

- ✓ users.js

(Les routes pour la page d'accueil)

(Les routes pour accès au front-end gérant les utilisateurs)

- ✓ **views**

- ✓ error.ejs

- %>))

- ✓ index.ejs

(HTML + Embedded JavaScript (<% your JavaScript code

(Fichier HTML de la page d'accueil + EJS)

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Fichier package.json :

```
{  
  "name": "blogapp",  
  "version": "0.0.0",  
  "private": true,  
  "scripts": {  
    "start": "node ./bin/www" },  
  "dependencies": {  
    "cookie-parser": "~1.4.4",  
    "debug": "~2.6.9",  
    "ejs": "~2.6.1",  
    "express": "~4.16.1",  
    "http-errors": "~1.6.3",  
    "morgan": "~1.9.1"  
  }  
}
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Fichier index.ejs :

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <title><%= title %></title>
```

```
    <link rel='stylesheet' href='/stylesheets/style.css' />
```

```
  </head>
```

```
  <body>
```

```
    <h1><%= title %></h1>
```

```
    <p>Welcome to <%= title %></p>
```

```
  </body>
```

```
</html>
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Fichier index.js :

```
var express = require('express');
```

```
var router = express.Router();
```

```
/* GET home page. */
```

```
router.get( '/', function(req, res, next) {  
    res.render('index', { title: 'Express' });  
}
```

```
);
```

```
module.exports = router;
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Fichier app.js :

```
var createError = require('http-errors');
var express = require('express');
var path = require('path');
var cookieParser = require('cookie-parser');
var logger = require('morgan');
var indexRouter = require('./routes/index');
var usersRouter = require('./routes/users');

var app = express(); //construct the application object
// view engine setup
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'ejs');
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Suite du fichier app.js :

```
app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());
app.use(express.static(path.join(__dirname, 'public')));
app.use('/', indexRouter);
app.use('/users', usersRouter);
// catch 404 and forward to error handler
app.use(function(req, res, next) {
    var err = new Error('Not Found'); err.status = 404; next(err);
});
// error handler ...
module.exports = app;
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Bibliothèques « express js » optionnelles :

- **compression** : compresser tous les « requests » : `app.use(compression())`
- **session** : création de sessions : `app.use(session({secret: 'Secr3t'}))`
- **method-override** : `app.use(methodOverride('_method'))`
- **response-time** : `app.use(responseTime())` ajoute le header « X-Response-Time » au « response ».
- **vhost** :
 - `app.use(vhost('*.user.local', userapp))`
 - `app.use(vhost('assets-*.example.com', staticapp))`
- **csrf** : protection contre le cross-site request forgery (CSRF) : `app.use(csrf())`;
- **timeout** :
 - `app.use(timeout('5s'))`;
 - `if (!req.timedout) next()`;

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

- ✓ Etablir la connexion entre « express JS » et « mongoDB » :
 - ✓ Ajouter le chemin de l'exécutable
 - ✓ Dans une « invite de commandes » indépendante, taper la commande suivante :
 > mongod
 - ✓ Dans « Node.js command prompt », sous le répertoire du projet « blogApp », taper la commande suivante :
 > npm install mongoose --save
 - ✓ Modifier le fichier « app.js » en lui ajoutant les lignes suivantes :

```
var mongoose = require('mongoose');
mongoose.connect('mongodb://localhost/blogApp', function(err) {
  if(err) {
    console.log('connection error', err);
  } else {
    console.log('connection successful');
  }
});
```
- ✓ Tester la connexion en lançant « npm start », le message « connection successful » s'affichera au log.

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

- ✓ Sous le répertoire du projet « blogApp », créer un dossier « models ».
- ✓ Sous le dossier « models », créer le fichier « blog.js » et taper dedans le code suivant :

```
var mongoose = require('mongoose');  
  
var blogSchema = new mongoose.Schema({  
  title: String,  
  author: String,  
  body: String,  
  comments: [{ author: String, body: String, date: Date }],  
  date: { type: Date, default: Date.now },  
  hidden: Boolean,  
  meta: { votes: Number, favs: Number }  
});  
  
module.exports = mongoose.model('Blog', blogSchema);
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Sous le dossier « models », créer le fichier « user.js » et taper dedans le code suivant :

```
var mongoose = require('mongoose');  
var userSchema = new mongoose.Schema({  
  name: String,  
  username: { type: String, required: true, unique: true },  
  password: { type: String, required: true },  
  admin: Boolean,  
  location: String,  
  meta: { age: Number, website: String },  
  created_at: { type: Date, default: Date.now },  
  updated_at: { type: Date, default: Date.now }  
});
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Suite du code du fichier « user.js » :

```
userSchema.methods.sayHello = function() {  
    return 'Hello' + this.name + '!';  
};  
userSchema.pre('save', function(next) {  
    var currentDate = new Date();  
    this.updated_at = currentDate;  
    if (!this.created_at)  
        this.created_at = currentDate;  
    next();  
});  
module.exports = mongoose.model('User', userSchema);
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Les routes en Express JS (REST Services)

Ressource : URI	POST (Create)	GET (Read)	PUT (Update)	DELETE (Destroy)
/blogs	Créer un nouveau billet	Lister tous les billets	N/A	N/A
/blogs/id	N/A	Consulter le billet	Modifier le billet	Supprimer le billet
/users	Ajouter un utilisateur	Lister les utilisateurs	N/A	N/A
/users/id	N/A	Afficher le profil	Modifier le compte	Supprimer le compte

requête HTTP « PUT » sur une URI suivant le format suivant : «
/users/id/changePassword »

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

- ✓ Sous le répertoire « routes » de l'application « blogApp », créer un fichier « blogs.js »
- ✓ Ainsi, les routes pour les billets du blog seront codées sur le nouveau fichier.
- ✓ Quant aux routes pour les utilisateurs, elles seront codées sur le fichier « users.js » qui est déjà créé sous le répertoire « routes ».
- ✓ Editer le fichier « app.js » se trouvant à la racine de l'application « blogApp » en y insérant ces lignes :

```
var blogs      = require('./routes/blogs');  
app.use('/blogs', blogs);
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

✓ Sous le répertoire « routes » de l'application « blogApp », éditer le fichier « blogs.js » :

```
var express = require('express');
var router = express.Router();

var mongoose = require('mongoose');
var Blog= require('../models/blog.js');

router.get('/',      function(req, res, next) {
                        Blog.find(      function (err, blogs) {
                                                if (err) return next(err);
                                                res.json(blogs);
                                            });
                    });

//Placer ici les autres requêtes HTTP gérées par le « router »

module.exports = router;
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

```
router.post('/', function(req, res, next) {  
  Blog.create(req.body, function (err, post) {  
    if (err) return next(err);  
    res.json(post);  
  });  
});  
  
router.get('/:id', function(req, res, next) {  
  Blog.findById(req.params.id, function (err, result) {  
    if (err) return next(err);  
    res.json(result);  
  });  
});
```

ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

```
router.put('/:id', function(req, res, next) {  
  Blog.findByIdAndUpdate(req.params.id, req.body, function (err, result) {  
    if (err) return next(err);  
    res.json(result);  
  });  
});  
router.delete('/:id', function(req, res, next) {  
  Blog.findByIdAndRemove(req.params.id, req.body, function (err, result) {  
    if (err) return next(err);  
    res.json(result);  
  });  
});
```

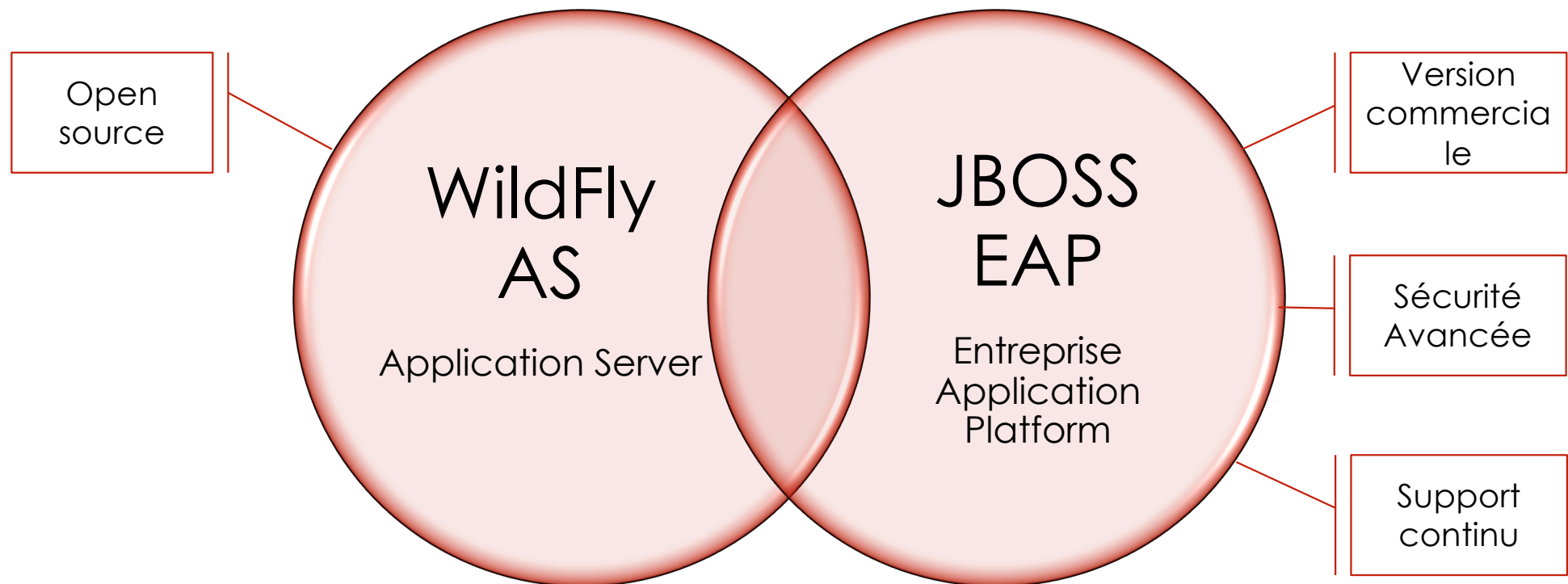
ETUDE DE CAS : NODE.JS + MONGOOSE JS + MONGODB

- ✓ Pour plus d'information sur l'API de mongoose JS, consulter la documentation officielle : <https://mongoosejs.com/docs/guide.html>
- ✓ Pour plus d'information sur l'API de routage d'express JS, consulter la documentation officielle : <https://expressjs.com/en/guide/routing.html>
- ✓ Pour utiliser les websockets : socket.io | ws
- ✓ Pour utiliser les protocoles CoAP et MQTT : Ponte : <https://www.eclipse.org/ponte/>
- ✓ Pour tester les services REST sans clients : Postman - REST Client (Plug-in Chrome)

ETUDE DE CAS : WILDFLY + HIBERNATE OGM + MONGODB

- ✓ Serveur d'application Jakarta ee 10
 - ✓ Fournisseur : RedHat
 - ✓ Serveur : WildFly 29.0.1.Final
 - ✓ Licence : Libre
- ✓ Communication avec MongoDB :
 - ✓ Jakarta NoSql
 - ✓ Jakarta Data
- ✓ Services REST : JAX-RS
- ✓ Websockets : WebSocket API (JSR 356)
- ✓ Sécurité : OAuth 2.0 Authentication Flow with Proof Key for Code Exchange (PKCE) + JWT
- ✓ Protocoles CoAP et MQTT : Californium (<https://eclipse.org/californium/>) et Paho (<http://eclipse.org/paho>)

ETUDE DE CAS : WILDFLY + HIBERNATE OGM + MONGODB

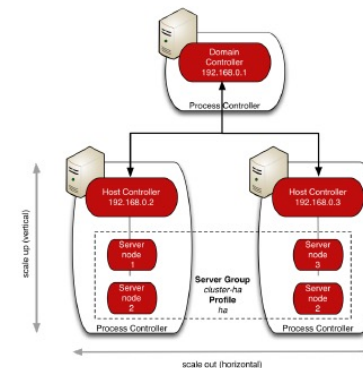


ETUDE DE CAS : WILDFLY + HIBERNATE OGM + MONGODB

- ✓ Architectures WildFly
 - ✓ Standalone
 - ✓ 1 seule JVM
 - ✓ Administré individuellement
 - ✓ Domain
 - ✓ Multi JVM, multi serveur
 - ✓ Notion de « host controller » et « domain controller »
 - ✓ Administration centralisée
 - ✓ Cloud en tant que PaaS
 - ✓ Microsoft Azure (Wildfly 29.0.1.Final) : <https://bitnami.com/stack/wildfly/cloud/azure>
 - ✓ RedHat Openshift: <https://www.mastertheboss.com/soa-cloud/openshift/using-wildfly-on-openshift/>

ETUDE DE CAS : WILDFLY + HIBERNATE OGM + MONGODB

- 3 types d'acteurs :
 - Des **serveurs WildFly** sur lesquels sont déployées les applications
 - Un « **host controller** » par serveur, chacun contrôlant les serveurs WildFly
 - Un « **domain controller** » permettant de piloter l'ensemble de l'environnement (cycle de vie, configuration, ...)



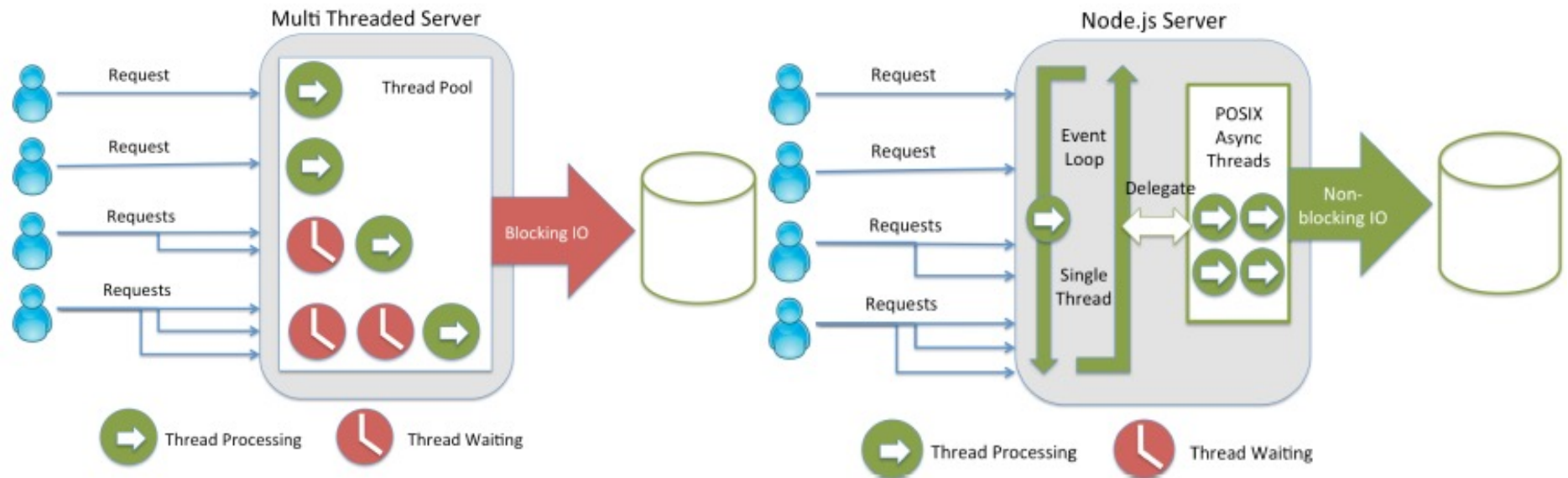
ETUDE DE CAS : WILDFLY + HIBERNATE OGM + MONGODB

- ✓ Mise en route de WildFly 29.0.1.Final :
 - ✓ Télécharger et décompresser l'archive
 - ✓ Déclaration de la variable d'environnement JBOSS_HOME (sera utilisé par Maven)
 - ✓ Vérification de votre JAVA_HOME
 - ✓ Lancement possible via le fichier « standalone.bat »
- ✓ Mise en route de Maven (Gestionnaire de paquetages en Java EE)
 - ✓ Télécharger l'archive des binaires de maven 3.6.1 :
<https://maven.apache.org/download.html>
 - ✓ Installer l'archive : <https://maven.apache.org/install.html>
- ✓ Voir « [Exercice 1](#) » pour la suite de l'étude de cas.

COMPARAISON ENTRE LES DEUX CAS SERVEUR

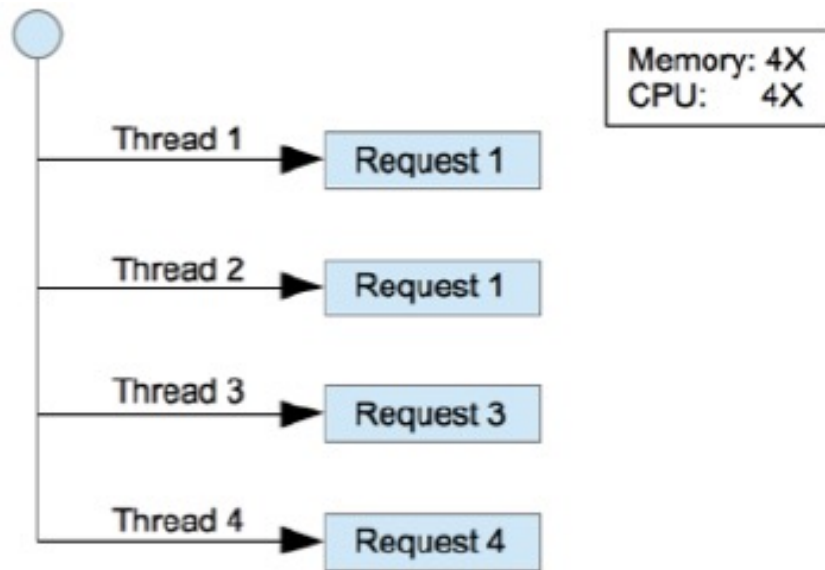
Fonctionnalité	Java EE	Node.js
Intégration EDI (IDE)	Oui (Avantage)	Oui
Gestion des dépendances	Maven, Ant	npm
Exécution	JVM via Serveur d'Applications	code binaire généré via JIT
Multithreading	Oui	Non
Montée en charge	Oui	Non
Sécurité	Oui (Avantage)	Oui
Modèle des Entrées/Sorties	Synchrone (Bloquant)	Asynchrone (Non Bloquant)
Distribution	Oui	Oui (Node cluster + zeroMQ via IPC)
Librairies standardisés	Oui	Non

COMPARAISON ENTRE LES DEUX CAS SERVEUR

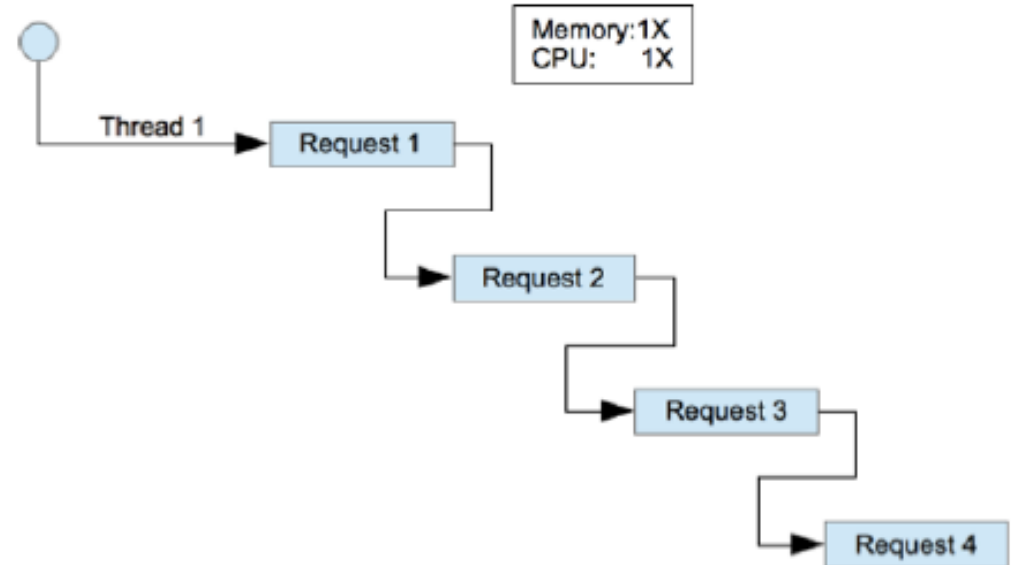


COMPARAISON ENTRE LES DEUX CAS SERVEUR

Java EE

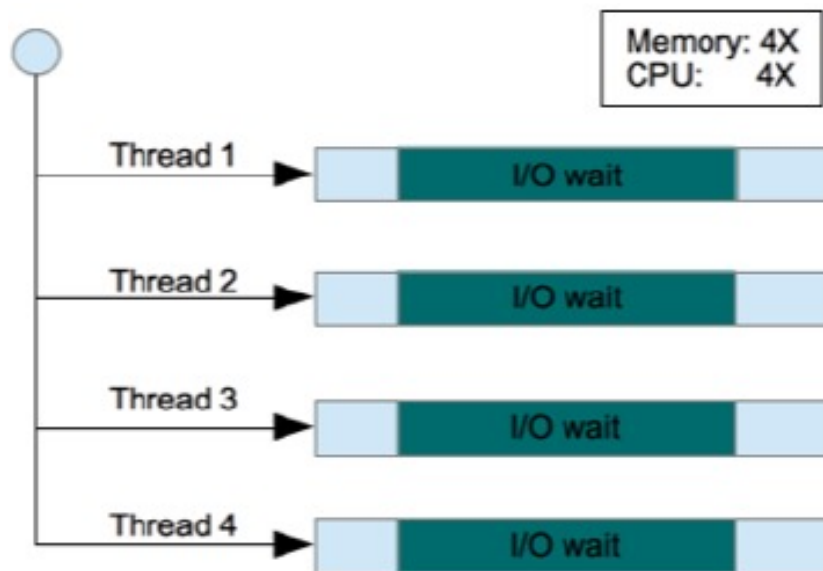


Node.js

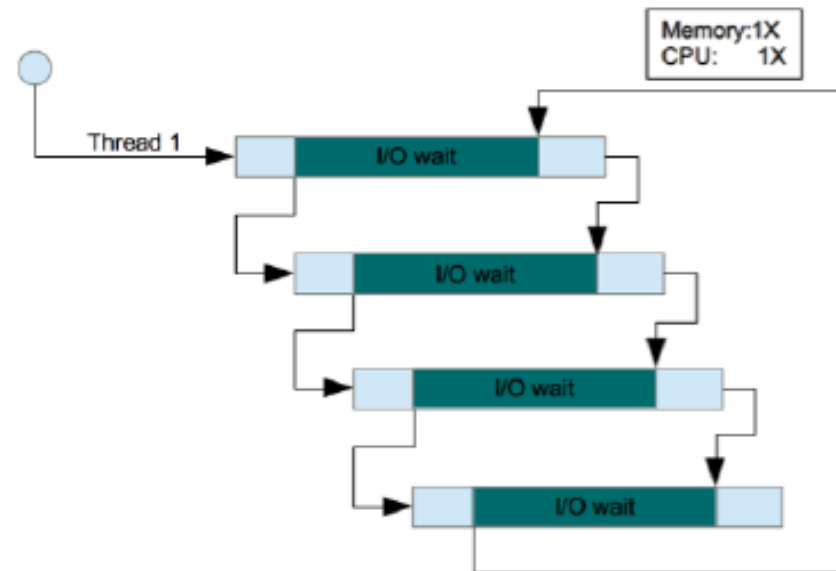


COMPARAISON ENTRE LES DEUX CAS SERVEUR

Java EE



Node.js



COMPARAISON ENTRE LES DEUX CAS SERVEUR

✓ Performances

- ✓ Node.js ne supporte pas les applications avec des requêtes nécessitant beaucoup de temps d'exécution sur le processeur.
- ✓ Dans l'IoT, la quasi-totalité des requêtes ne demandent pas du temps processeur mais plutôt des E/S → Node.js est adapté à l'IoT
- ✓ **Les deux solutions serveurs se valent en termes de performances.**
- ✓ Cependant, Node.js nécessite d'être configuré avec « Node Cluster » et une couche distribuée de communication interprocessus (IPC) utilisant par exemple zeroMQ pour assurer de bonnes performances en production.
- ✓ La scalabilité et la montée en charge des serveurs Java EE ainsi que leur relative facilité de configuration en mode distribué sont des avantages certains en production.

CLOUD OF THINGS

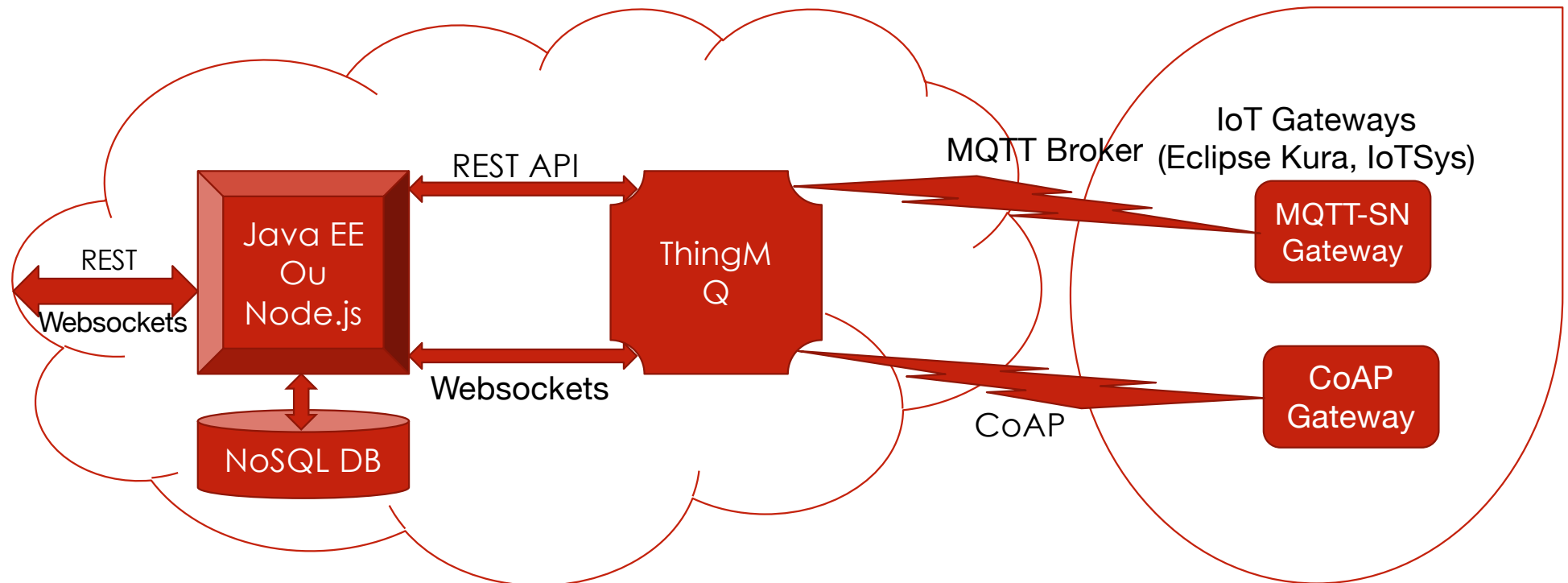
✓ Cloud of Things

- ✓ Serveur Middleware hébergé sur le Cloud (PaaS) : Java EE Server ou Node.js Server
- ✓ Communication avec les « Things » via un courtier multi-protocole sur le Cloud (p.ex. ThingMQ : <https://thingmq.com>)
- ✓ Un « IoT integration middleware » (Eclipse Kura ou Automation Systems Group IoT SyS) pour :
 - ✓ En développement : simuler les « Things »
 - ✓ En production : connecter les « Things » réels au courtier multi-protocole sur le Cloud (Gateways)

✓ Alternatives:

- ✓ Flogo : <https://www.flogo.io>
- ✓ Node-Red : <https://nodered.org>

CLOUD OF THINGS



CLOUD OF THINGS

